

LI-ESKG: Latent Identities in Event-State Knowledge Graphs

A. CN
acn@gravitalia.com

Abstract

Modern systems rely on heterogeneous observations from cameras, microphones, textual documents, geolocation sensors, and machine learning pipelines to construct evolving representations of the physical world. Event-State Knowledge Graphs (ESKGs) provide an expressive framework for modeling temporal evolution and causal relationships between events and states. However, they implicitly assume that the identity of the underlying entities is already known. We introduce latent identity on ESKG (LI-ESKG), an ESKG extension that models observations, latent identities, and probabilistic associations while preserving the causal semantics of event-state transitions.

Keywords

Entity Resolution, Knowledge Graph, Belief Propagation, Latent Identities

1 Introduction

Knowledge Graphs (KGs) provide a structured, semantically rich representation of human knowledge. They depict “an integrated collection of real-world entities connected by semantically interrelated relations” [1]. This enables advanced reasoning, semantic querying, and information integration across heterogeneous domains. To capture temporal dynamics, Event-State Knowledge Graphs (ESKGs) extend this static paradigm through an event-driven representation [13]. In an ESKG, events trigger explicit state transitions, maintaining a continuous historical record of the world’s evolution.

This approach improves causal reasoning, but it assumes that the identity of each state and event is already uniquely resolved. This assumption rarely holds in perception-driven systems. Autonomous agents, cyber-physical systems, and ambient intelligence platforms do not observe abstract entities directly. Instead, they receive a continuous stream of raw, heterogeneous, and noisy observations from various sensors and downstream machine learning models. For example, cameras produce face embeddings, microphones yield speaker embeddings, OCR extract text, and GPS receivers estimate coordinates.

Each observation is partial, uncertain, and modality-specific. In this context, physical identities are not explicit inputs; they are latent variables that must be inferred by integrating disparate observations over space and time. Entity Resolution (ER), or record linkage, addresses this problem by determining if multiple observations describe the same real-world entity [3, 4]. Modern ER frameworks combine supervised learning [7], Bayesian inference [8, 9], collective reasoning over relational networks [2], and heterogeneous representation learning [10]. In industrial contexts, tools like Splink and Dedupe scale these matching algorithms to massive datasets.

Beyond pairwise matching, knowledge graph research utilizes ontology matching, entity alignment, and Graph Neural Networks (GNNs) to align separate knowledge bases using graph topology

[1, 11]. However, these approaches treat identity inference as an external, one-time pre-processing step. Existing graph formalisms assume that entities exist *a priori* as explicit nodes. They lack the native syntax and semantics to represent identity uncertainty or to model identity formation as an ongoing, first-class process.

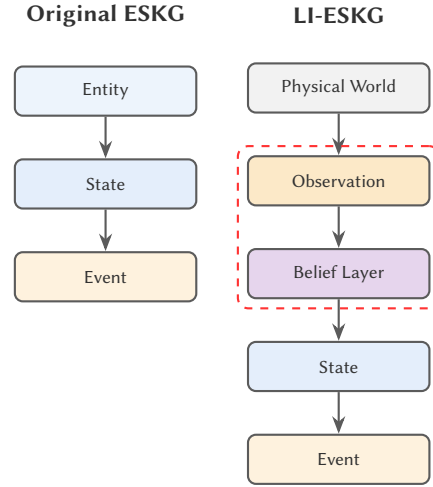


Figure 1: Differences between ESKG and LI-ESKG execution paradigms.

This limitation is fundamentally ontological. Observations and identities belong to distinct categories and different levels of abstraction. Observations are empirical, immutable evidence collected at specific coordinates in space and time. By contrast, identities are latent explanatory hypotheses constructed to explain why those observations occurred. This perspective aligns with probabilistic graphical models, where latent variables explain observed data [5]. Conflating these concepts forces a knowledge graph to represent speculative, inferred hypotheses as unquestionable facts. To support reliable reasoning, representations must separate mutable, uncertain identity hypotheses from immutable, empirical evidence (see Fig. 1).

To address this, we introduce the *Latent Identity Event-State Knowledge Graph* (LI-ESKG). LI-ESKG is an observation-centric framework that decouples four core concepts: observations, events, states, and identity hypotheses. Observations form the immutable foundation of the graph, while events and states model temporal evolution. Identity hypotheses are maintained within a dedicated, ephemeral mathematical construct called the *Belief Layer*, separating the persistent knowledge representation from the computational overhead of online reasoning.

This paper establishes the formal, mathematical, and practical execution foundations of the LI-ESKG model through four main contributions. First, we define an observation-centric ontology that

represents identity hypotheses as first-class graph objects, separating empirical evidence from inferred entities. Second, we introduce an intermediate mathematical *Belief Layer* that maintains bounded statistical summaries of active entities, removing raw observations from the real-time inference loop. Third, we design a formal *Execution Model* structured as a state machine with transactional primitives to guarantee runtime invariants. Finally, we provide a strongly typed reference implementation in Rust leveraging an Entity Component System (ECS) architecture, demonstrating how complex multi-modal observation models can be composed under uniform probabilistic semantics.

2 Related Work

Identity reasoning in dynamic environments spans several research fields. Table 1 compares how these fields handle observations, identities, causality, and persistence.

Table 1: Comparison of LI-ESKG with related paradigms.

Model	Observations	Identity	Causality	Persistent KG
Classical ER	✓	Latent	×	×
Knowledge Graph	×	Explicit	×	✓
Temporal KG	×	Explicit	Implicit	✓
ESKG	×	Explicit	✓	✓
PGMs	✓	Latent	×	×
LI-ESKG	✓	Latent	✓	✓

Entity Resolution. Classical Entity Resolution (ER) matches physical observations using statistical decisions [3, 4]. Bayesian models extend this by representing identity uncertainty over latent entities to resolve matches jointly [8, 9]. Collective ER exploits relational dependencies between records instead of assuming independent pairwise comparisons [2]. Deep representation learning enables cross-domain and multimodal matching by projecting heterogeneous observations into shared embedding spaces [7, 10]. However, ER systems typically run as external, offline pre-processing pipelines. They output static decisions, meaning the resulting knowledge graph cannot dynamically represent or update identity uncertainty.

Knowledge Graphs. Knowledge Graphs (KGs) represent structured facts and semantic relations to support reasoning and queries [1]. Temporal KGs add timestamps or intervals to edges, while dynamic KGs allow the graph topology to evolve. To align different graphs, entity and ontology matching methods use Graph Neural Networks (GNNs) to evaluate structural similarity [11]. However, these approaches assume that entities exist as explicit, predefined nodes. Although models can represent uncertainty on edges, attributes, or alignment links, they do not treat entity existence itself as a latent, uncertain variable.

Event-State Knowledge Graphs. Event-State Knowledge Graphs (ESKGs) model physical causality by using events to trigger explicit state transitions [13]. This causal representation models graph evolution through explicit transitions rather than independent temporal snapshots. However, ESGGs assume that every state belongs to a pre-resolved, known entity. They track how established entities evolve over time, but they cannot represent how these entities are constructed or identified from raw, noisy sensor data.

Probabilistic Graphical Models. Probabilistic Graphical Models (PGMs), such as Bayesian Networks and Markov Random Fields, model joint probability distributions using conditional dependencies [5]. Factor graphs generalize PGMs by decomposing global distributions into local functions [6], enabling efficient marginal estimation via Belief Propagation [12]. While PGMs effectively model latent variables and observation uncertainty, they are designed for short-term inference rather than long-term knowledge representation. They lack a structured semantic ontology for events and states, and they do not maintain a persistent, causal history of the world.

Positioning. As shown in Table 1, LI-ESKG combines these paradigms. It provides an execution framework where observations serve as immutable evidence, active identities are resolved in a transient Belief Layer, and states evolve via causal transitions. By isolating online inference within this Belief Layer, LI-ESKG combines the uncertainty management of PGMs with the persistent, causal tracking of ESGGs.

Inference

Since the ontology separates raw observations from latent identities, incoming observations introduce identity uncertainty that prevents deterministic graph updates. We resolve this uncertainty using probabilistic inference within the ephemeral Belief Layer instead of modifying the persistent graph directly.

3.1 Operational Pipeline

The operational pipeline decouples persistent storage from inference overhead through a sequential processing cycle. Upon receiving an observation o_t , the system updates its state according to the following sequence:

$$G_t \xrightarrow{\Phi^{-1}} B_t \xrightarrow{\Psi} F_t \xrightarrow{BP} P(Z_t) \xrightarrow{\Gamma} B_{t+1} \xrightarrow{\Phi} G_{t+1}.$$

The inverse commitment mapping Φ^{-1} first isolates active neighborhoods. The factor graph compiler Ψ then constructs a localized factor graph $F_t = \Psi(B_t, o_t)$ over candidate assignment variables. Belief Propagation (BP) computes the exact marginal posteriors $P(Z_t)$. Finally, the belief state is updated via Γ , and the resulting structural updates are committed to the persistent graph via Φ . Algorithm 1 outlines this workflow.

Algorithm 1: LI-ESKG Mathematical Execution Pipeline

Input: Incoming observation o_t , Current System State (G_t, B_t)

Output: Updated System State (G_{t+1}, B_{t+1})

- 1 $B_t \leftarrow \text{ExtractActiveBeliefs}(G_t, o_t)$
 - 2 $F_t \leftarrow \Psi(B_t, o_t)$
 - 3 $P(Z_t) \leftarrow \text{RunBeliefPropagation}(F_t)$
 - 4 $Z^* \leftarrow \arg \max_Z P(Z_t)$
 - 5 $B_{t+1} \leftarrow \Gamma(B_t, P(Z_t))$
 - 6 $G_{t+1} \leftarrow \Phi(B_{t+1}, Z^*)$
 - 7 **return** (G_{t+1}, B_{t+1})
-

3.2 Factor Graph Construction

To leverage conditional independencies, the system constructs a transient factor graph $F = (X, \Phi)$ following the topological decoupling shown in Fig. 2. The variable nodes $X = \{Z_1, \dots, Z_n\}$ correspond to discrete assignment variables.

For each observation o_j , we define a discrete random variable $Z_j \in I_q \cup \{\perp\}$, where $Z_j = i$ denotes assignment to identity hypothesis i and $Z_j = \perp$ triggers the creation of a new identity. The joint posterior probability factorizes over local compatibility functions:

$$P(Z | O) = \frac{1}{Z_G} \prod_{\phi \in \Phi} \phi(Z_\phi),$$

where Z_G is the normalizing partition function. The compatibility factors evaluate statistical dependencies across four distinct domains: biometric similarity using high-dimensional embeddings θ_i , temporal consistency via time gaps in Λ_i , spatial consistency via motion dynamics under covariance Σ_i , and semantic compatibility via metadata rules.

3.3 Belief Propagation

We compute marginal posteriors using the Sum-Product Belief Propagation algorithm [12], as illustrated in Fig. 3. Variable nodes and factors iteratively exchange messages:

$$m_{\phi \rightarrow Z_j}(Z_j) = \sum_{Z_\phi \setminus \{Z_j\}} \phi(Z_\phi) \prod_{Y \in \text{ne}(\phi) \setminus \{Z_j\}} m_{Y \rightarrow \phi}(Y).$$

Upon convergence, the marginal distributions:

$$P(Z_j = i | O) \propto \prod_{\phi \in \text{ne}(Z_j)} m_{\phi \rightarrow Z_j}(i)$$

provide the exact probability scores required by the update operator Γ .

3.4 Identity Transitions

The commitment operator Φ resolves the MAP configurations into specific structural modifications in G_{t+1} . If $Z_q^* = \perp$, the system executes identity creation by adding a new node $i_{\text{new}} \in I$ to V and initializing its belief summary in B_t . If $Z_q^* = i$, the system performs support augmentation by linking the observation via the relation $(o_q, \text{supports}, i) \in R$ and updating b_i . If the configuration triggers an identity collision between two nodes i_1 and i_2 , the system performs identity refinement (merge) to combine their histories into a single representative node while preserving all historical event-state relations. Algorithm 2 details this transactional sequence.

4 Theoretical Analysis

This section proves the structural soundness, causal consistency, and space-time bounds of the LI-ESKG framework.

THEOREM 1 (CONSERVATIVE EXTENSION). *Every Event-State Knowledge Graph admits an embedding into a LI-ESKG.*

PROOF. Let $G_{\text{eskg}} = (E, S, R_{\text{eskg}})$ be an arbitrary Event-State Knowledge Graph. For every represented entity in G_{eskg} , we construct exactly one observation node $o \in O$ and one identity node $i \in I$ in the LI-ESKG. We connect them via a single relation $(o, \text{supports}, i) \in R$. Because each identity node possesses a singleton support set

Algorithm 2: Graph Topologies Materialization (Φ)

Input: Graph G_t , MAP assignment Z^* , observation o_q

Output: Updated Graph G_{t+1}

```

1 if  $Z_q^* = \perp$  then
2    $i_{\text{new}} \leftarrow \text{CreateNewIdentityNode}()$ 
3    $G.V \leftarrow G.V \cup \{i_{\text{new}}\}$ 
4    $G.R \leftarrow G.R \cup \{(o_q, \text{supports}, i_{\text{new}})\}$ 
5 else
6    $i_{\text{target}} \leftarrow Z_q^*$ 
7    $G.R \leftarrow G.R \cup \{(o_q, \text{supports}, i_{\text{target}})\}$ 
8   if  $\text{IdentifyMergeRequired}(i_{\text{target}})$  then
9      $i_{\text{dup}} \leftarrow \text{GetDuplicateNode}(i_{\text{target}})$ 
10    for  $o \in \text{supp}(i_{\text{dup}})$  do
11       $G.R \leftarrow (G.R \setminus \{(o, \text{supports}, i_{\text{dup}})\}) \cup$ 
12         $\{(o, \text{supports}, i_{\text{target}})\}$ 
13    end
14    for  $(i_{\text{dup}}, \lambda, v) \in G.R$  do
15       $G.R \leftarrow (G.R \setminus \{(i_{\text{dup}}, \lambda, v)\}) \cup \{(i_{\text{target}}, \lambda, v)\}$ 
16    end
17     $G.V \leftarrow G.V \setminus \{i_{\text{dup}}\}$ 
18  end
19 return  $G$ 

```

and no identity uncertainty exists, all structural invariants of LI-ESKG are trivially satisfied. Removing the additional observation and identity layers recovers G_{eskg} without altering its event-state transitions. $\square$

THEOREM 2 (PROJECTION PRESERVATION). *The projection mapping $\pi : (O, I, E, S) \rightarrow (E, S)$ is a graph homomorphism preserving every causal relation defined by ESKG.*

PROOF. The projection operator π deletes all observation nodes O , identity nodes I , and relations in R_{LI} . Causal relations inherited from the ESKG schema remain unchanged because their endpoints lie strictly within $E \cup S$. Consequently, any causal path in the Event-State subgraph is preserved under π , proving that π is a valid graph homomorphism. $\square$

THEOREM 3 (IDENTITY UNIQUENESS). *At every instant, two distinct active identities cannot represent the same physical entity.*

PROOF. By definition, the physical mapping $\gamma_t : I_t \rightarrow \Omega_t$ is bijective. Assume there exist two distinct active identity hypotheses $i_1, i_2 \in I_t$ such that $i_1 \neq i_2$, but both represent the same physical entity $\omega \in \Omega_t$. This implies $\gamma_t(i_1) = \gamma_t(i_2) = \omega$, which directly violates the injectivity of the bijection γ_t . Thus, identity uniqueness holds. $\square$

THEOREM 4 (OBSERVATION PARTITION). *The support sets of active identities partition the active observations.*

PROOF. By construction, every active observation $o \in O_{\text{active}}$ must support exactly one active identity hypothesis $i \in I_{\text{active}}$. Conversely, every active identity node i must have a non-empty

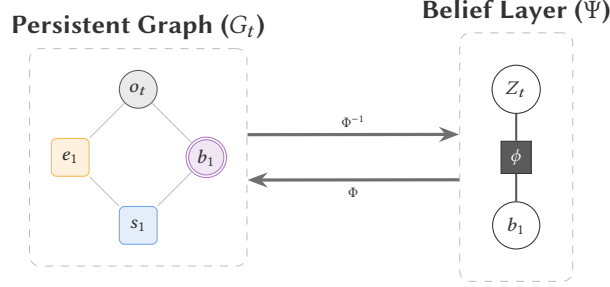


Figure 2: Operational decoupling between persistent knowledge graph and the Belief Layer execution pipeline.

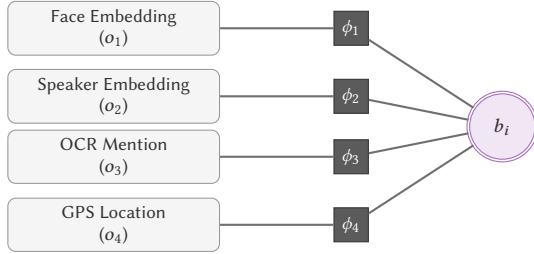


Figure 3: Factor graph configuration for multi-modal identity tracking.

support set $\text{supp}(i)$. Therefore, the support sets are disjoint, and their union recovers the active observation set:

$$O_{\text{active}} = \bigsqcup_{i \in I_{\text{active}}} \text{supp}(i).$$

□

THEOREM 5 (CAUSAL INDEPENDENCE). *Identity inference preserves the causal semantics of the Event-State subgraph.*

PROOF. The identity inference operator $\mathcal{T}_I$ only modifies vertices in $O \cup I$ and relations in R_{LI} . It does not alter events E , states S , or causal transition relations in R_{eskg} . Applying the projection operator π yields:

$$\pi \circ \mathcal{T}_I = \pi,$$

proving that identity inference operations leave the causal Event-State component invariant. □

THEOREM 6 (LOCALITY OF INFERENCE). *Let $\mathcal{G}$ be the global factor graph representing the joint probability distribution $P(Z \mid O)$. Let $\mathcal{B}(Z_q) \subseteq V$ be the Markov blanket of the query assignment variable Z_q . If the extracted local neighbourhood satisfies $\mathcal{B}(Z_q) \subseteq \mathcal{N}(o_q)$, then probabilistic inference restricted to the temporary factor graph F constructed over $\mathcal{N}(o_q)$ yields the exact global marginal posterior for Z_q :*

$$P(Z_q \mid O) = P(Z_q \mid O_{\mathcal{N}(o_q)}).$$

PROOF. Let Z_N denote the set of assignment variables within the local neighbourhood $\mathcal{N}(o_q)$, and let $Z_{\setminus N}$ denote the remaining variables in the global graph. By definition, the Markov blanket $\mathcal{B}(Z_q)$ d-separates Z_q from all variables in $Z_{\setminus N}$. Since $\mathcal{B}(Z_q) \subseteq$

$\mathcal{N}(o_q)$, the query variable Z_q is conditionally independent of $Z_{\setminus N}$ given the remaining local variables:

$$Z_q \perp Z_{\setminus N} \mid (Z_N \setminus \{Z_q\}).$$

The global joint distribution factorizes into local compatibility functions:

$$P(Z \mid O) = \frac{1}{Z_G} \prod_{\phi \in \Phi_N} \phi(Z_\phi) \prod_{\psi \in \Phi_{\setminus N}} \psi(Z_\psi),$$

where Φ_N is the set of factors containing only variables in Z_N , and $\Phi_{\setminus N}$ contains factors with at least one variable in $Z_{\setminus N}$. Because $\mathcal{B}(Z_q) \subseteq \mathcal{N}(o_q)$, no factor in $\Phi_{\setminus N}$ contains Z_q . Marginalizing out the external variables $Z_{\setminus N}$ yields:

$$P(Z_N \mid O) = \sum_{Z_{\setminus N}} P(Z \mid O) = \frac{1}{Z_G} \prod_{\phi \in \Phi_N} \phi(Z_\phi) \sum_{Z_{\setminus N}} \prod_{\psi \in \Phi_{\setminus N}} \psi(Z_\psi).$$

The summation term acts as a local prior over the boundary variables of $\mathcal{N}(o_q)$ and is independent of Z_q . Consequently, local inference restricted to the factor graph $F = (X_N, \Phi_N)$ preserves the exact relative state probabilities of Z_q . Normalizing this local distribution yields $P(Z_q \mid O) = P(Z_q \mid O_{\mathcal{N}(o_q)})$. □

THEOREM 7 (BOUNDED MEMORY SPACE). *Let $M = |I_{\text{active}}|$ be the count of concurrent active identities tracked within the system. Under the bounding history constraint $|\Lambda_i| \leq K$, the memory footprint of the ephemeral Belief Layer is strictly bounded by $O(M)$, independent of the total accumulated historical observations $|O|$.*

PROOF. Each individual active belief state $b_i = (\theta_i, \Sigma_i, \Lambda_i)$ has a structural memory allocation size bounded by $|b_i| = |\theta_i| + |\Sigma_i| + |\Lambda_i|$. Given that θ_i and Σ_i have fixed continuous spatial dimensions, their space usage evaluates to $O(1)$. Because the history buffer size is bounded by $|\Lambda_i| \leq K$, the size of a single belief node resolves to $|b_i| \leq c + K = O(1)$ for a fixed constant c . Summing across the active partition yields the total space complexity:

$$|B_t| = \sum_{i=1}^M |b_i| \leq M \cdot O(1) = O(M).$$

This proves the space requirements depend exclusively on current active tracking scales, discarding historic observation volumes. □

5 Reference Implementation

We have done the mathematical design of the LI-ESKG framework through a reference implementation in Rust. The architecture maps formal boundaries into compile-time invariants.

5.1 Typed Ontology

The partitioned vertex set $V = O \sqcup I \sqcup E \sqcup S$ is enforced as an algebraic data type (ADT), eliminating out-of-schema entries at compile time.

Listing 1: Vertex ontology and semantic relations.

```
pub enum Vertex {
    Observation(Observation),
    Identity(Identity),
    Event(Event),
    State(State),
}

pub enum Relation {
    // Event-State
    Trigger,
    Lead,
    Evolution,
    Contain,
    Occur,
    Influence,

    // Identity-Belief
    ObservedDuring,
    Describes,
    Supports,
    Refines,
    AssociatedWith,
}
```

5.2 Belief State Enforcements

To implement the mathematical separation of rolling statistics from structural graph definitions, we separate identity objects from their probabilistic representations.

Listing 2: Belief layer type mappings.

```
pub struct Identity {
    pub id: IdentityId,
}

pub struct BeliefState<M: ObservationModel> {
    pub identity: IdentityId,
    pub model: M,
}
```

Heterogeneous tracking strategies are bound to a uniform execution standard by utilizing an abstract `ObservationModel` trait interface. This abstraction handles likelihood computations, state mutations, and historical memory management.

Listing 3: Observation model trait specification.

```
pub trait ObservationModel: Send + Sync {
    type Summary;
```

```
fn likelihood(&self, payload: &Payload) -> f64;
fn merge(&mut self, other: &Self);
fn decay(&mut self, delta_t: f64);
fn checkpoint(&self) -> Vec<u8>;
}
```

The decay function incorporates system-level forgetting thresholds, allowing identity state representations to gracefully down-weight historic observations over long time intervals.

5.3 Entity Component System (ECS) Workspace

To execute multi-modal integration efficiently, the implementation employs an Entity Component System (ECS) architecture pattern. The system engine acts as a unified Workspace where independent tracking modalities are decoupled into separate components that implement the uniform `ObservationModel` interface.

Listing 4: ECS Workspace and Component architecture.

```
pub struct Workspace {
    pub beliefs: HashMap<IdentityId, Vec<Box<dyn
        ObservationModel>>>,
}

pub struct FaceComponent {
    pub embedding: Vec<f32>,
}

pub struct GpsComponent {
    pub coordinates: (f64, f64),
    pub covariance: [[f64; 2]; 2],
}

pub struct TextComponent {
    pub mentions: Vec<String>,
}

impl ObservationModel for FaceComponent {}
impl ObservationModel for GpsComponent {}
impl ObservationModel for TextComponent {}
```

This setup yields dynamic composition. If an identity is tracked concurrently via video streams and GPS sensors, its tracking footprint expands at runtime by appending both `FaceComponent` and `GpsComponent`. The inference pipeline remains completely generic, querying the underlying component vector under unified probabilistic semantics.

5.4 Performance Invariants

The decoupled Belief Layer isolates high-throughput operational workloads from the underlying persistent storage. Real-time execution is governed by three primary operational bounds:

$$\text{Latency} = O(K)$$

$$\text{Memory} = O(|I_{\text{active}}|)$$

$$\text{Recovery} = O(|L| + \Delta O)$$

where K is the history buffer limit, $|I_{\text{active}}|$ is the number of active identities, $|L|$ is the size of the latest serialized checkpoint, and ΔO is the volume of raw observations received since the last checkpoint.

5.5 Crash Recovery and Checkpointing

Fault tolerance relies on a non-blocking state serialization. Incoming observations are written to an append-only transaction log. Periodically, the Checkpoint primitive spawns background threads to serialize the compact BeliefState summaries via the checkpoint() interface.

Following a crash, the Recover routine restores the latest Belief Layer snapshot and replays only the observation delta ΔO . Consequently, recovery time depends on the log delta rather than the total historical size of the knowledge graph.

6 Discussion

The formal decoupling of observations and identities within a unified execution model offers several key structural advantages. First, LI-ESKG provides a complete execution framework rather than an isolated ontology or standalone matching model. By defining the ontology, semantics, inference theory, and execution model under hardware constraints, the framework mirrors systems like Apache Beam (unifying semantics with runtime windows) or GTSAM (coupling factor graphs with state estimation). Second, anchoring events and states to latent variables in the Belief Layer extends causal expressiveness to noisy, perception-driven environments without degrading the underlying state-transition logic. Third, the core architecture remains solver-independent, allowing the replacement of factor graphs and Belief Propagation with Bayesian networks or differentiable neural models by extending the ObservationModel trait. Finally, the reference implementation confirms that structural invariants—such as vertex constraints—can be validated at compile time, eliminating runtime schema enforcement.

7 Conclusion

We introduced the Latent Identity Event-State Knowledge Graph (LI-ESKG), a formal framework that decouples physical observations from identity hypotheses via an intermediate mathematical Belief Layer. By modeling identity resolution as a continuous, real-time execution process rather than a static pre-processing step, the framework supports joint uncertainty management and causal reasoning. Supported by formal execution primitives, bounded space-time guarantees, and an extensible Rust ECS implementation, LI-ESKG provides a scalable architecture for knowledge representation in noisy, dynamic environments.

References

- [1] Bilal Abu-Salih. 2021. Domain-specific Knowledge Graphs: A Survey. *Journal of Network and Computer Applications* 192 (2021), 103076. doi:10.1016/j.jnca.2021.103076
- [2] Indrajit Bhattacharya and Lise Getoor. 2007. Collective entity resolution in relational data. *ACM Trans. Knowl. Discov. Data* 1, 1 (March 2007), 5–es. doi:10.1145/1217299.1217304
- [3] Peter Christen. 2012. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, New York, NY.
- [4] Ivan Fellegi and Alan Sunter. 1969. A Theory for Record Linkage. *J. Amer. Statist. Assoc.* 64, 328 (1969), 1183–1210. doi:10.1080/01621459.1969.10501049
- [5] Daphne Koller and Nir Friedman. 2009. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.
- [6] Frank R. Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. 2001. Factor Graphs and the Sum-Product Algorithm. In *Proceedings of the IEEE*. Vol. 47. 498–519. doi:10.1002/0471748602.ch6
- [7] Ramon Pereira, Wagner Junior, and Augusto Afonso Guerra Junior. 2024. The use of Supervised Learning to perform pairwise classification for Record Linkage over real world data. *International Journal of Population Data Science* 9, 5 (Sep. 2024). doi:10.23889/ijpds.v9i5.2902
- [8] Rebecca C. Steorts, Rob Hall, and Stephen E. Fienberg. 2015. A Bayesian Approach to Graphical Record Linkage and De-duplication. arXiv:1312.4645 [stat.ME] <https://arxiv.org/abs/1312.4645>
- [9] Andrea Tancredi and Brunero Liseo. 2011. A Hierarchical Bayesian Approach to Record Linkage and Population Size Problems. *Annals of Applied Statistics* 5, 2B (2011), 1553–1585. doi:10.1214/10-AOAS447
- [10] Yunong Tian, Ning Wang, and Anshun Zhou. 2026. CrossER: A robust and adaptable generalized entity resolution framework for diverse and heterogeneous datasets. *Information Systems* 135 (2026), 102609. doi:10.1016/j.is.2025.102609
- [11] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S. Yu. 2019. Heterogeneous Graph Attention Network. In *Proceedings of the WWW Conference 2019*. 2022–2032. doi:10.1145/3308558.3313562
- [12] Jonathan S. Yedidia, William T. Freeman, and Yair Weiss. 2003. Understanding Belief Propagation and Its Generalizations. *Exploring Artificial Intelligence in the New Millennium* 8 (2003), 239–269.
- [13] T. Zang, Y. Li, M. Yang, and P. Jiang. 2026. Event-state knowledge graph-enhanced GraphRAG for order-driven 3D printing supply chain configuration. *Expert Systems with Applications* 317 (2026), 121401–121415. doi:10.1016/j.eswa.2025.121401